

Web Semântica

Experimentação / Resultados / Conclusão / Trabalhos
Futuros

Pedro Saito

Universidade Federal do Rio de Janeiro

2026-07-08



UFRJ

1. Experimentação

Construção e análise de motores de consultas SPARQL federadas, avaliado pelo benchmark do FedShop.

Engine	Fontes	Join	Escalab.	Pré-proc.	Cardinalidade
RSA	Broadcast de SERVICE	Hash join local	Alta	Não	–
go-fed	ASK + cache	Hash / bind join	Média	Não	–
FedX	ASK + cache	Bound join	Alta	Não	<i>join order</i>
PyFedX	ASK	Hash join local	Média	Não	–
SPLENDID	Índice VoID	Hash join	Alta	Sim	<i>triple count</i> + partições
ANAPSID	Catálogo LAV	agjoin adaptativo	Média	Parcial	stats + <i>sampling</i>
SemaGrow	SPARQLED / VoID	Hash join	Alta	Sim	Custo com cardin.

Tabela 1: *Comparação dos motores de consultas avaliados.*

Dimensão	Métricas	Pergunta respondida
Correção	<code>nb_results, mismatch</code>	A resposta equivale à referência?
Tempo	<code>exec_time,</code> <code>source_selection_time,</code> <code>planning_time</code>	Onde o tempo é consumido?
Rede	<code>ask, http_req,</code> <code>data_transfer</code>	Quanto custa consultar a federação?
Seleção	<code>tpwss,</code> <code>nb_distinct_sources,</code> <code>relevant_sources_selectivity</code>	Fontes irrelevantes selecionadas?
Robustez	<code>is_timeout, is_error</code>	O plano é estável sob carga?

2. Resultados

Cenário: Usuário em uma loja virtual composta por:

Produtos *Vendedores* *Sites de avaliação*

Consultas: Três categorias de consultas federadas:

Single Domain (SD): TP de um único *endpoint*

Multi Domain (MD): TP de vários *endpoints* sem junção

Cross-Domain (CD): TP de diversos *endpoints* com junção

Federações sintéticas variando de 20 – 200 membros.

Cada federação com seu próprio *endpoint* SPAQL e URIs.

Entidades locais ligadas ao catálogo global por owl:sameAs.

Catálogo Virtual

Product → **Producer**

Product → **ProductType**

Product → **ProductFeature**

Vendedores

Offer → **Product**

Offer → **Vendor**

Sites de Avaliação

Review → **Product**

Review → **Person**

Review → **RatingSite**

Tipo	Motor	exec_time	planning_time	source_selection_time	join_time
Oráculo	RSA	9,40	1,23	1,23	8,17
	FedShop-Go	0,44	0,00	0,06	0,38
	PyFedX	9,21	0,00	0,40	8,81
	PyFedX++	2,03	0,00	0,71	1,32
Baseline	FedX	3,87	0,00	0,14	3,74
Metadados	SPLENDID	13,67	—	—	—
	SemaGrow	67,10	—	—	—

Tabela 2: Tempos médios (em segundos) por estágio. Dourado indica melhor global da coluna.

Tipo	Motor	Qualidade			Erros					OK
		Precisão	Revocação	F1	Espúrios	Faltantes	Duplicatas	Ausentes	Mismatch (%)	
Oráculo	RSA	90,9%	90,9%	90,9%	4	4194304	0	0	9,1%	91,7%
Implementa- ção	FedShop-Go	100,0%	100,0%	100,0%	0	0	0	0	0,0%	83,3%
	PyFedX	100,0%	100,0%	100,0%	0	0	0	0	0,0%	60,4%
	PyFedX++	80,0%	100,0%	80,0%	80	0	36	0	20,0%	83,3%
Baseline	FedX	100,0%	37,5%	37,5%	0	4194455	0	188	62,5%	100,0%
Metadados	SPLENDID	100,0%	98,9%	99,3%	0	3	0	0	2,5%	75,5%
	SemaGrow	89,7%	59,1%	59,1%	30	118	0	110	40,9%	91,7%

Figura 3: Métricas de corretude agregadas por motor; Verde indica melhor global.

Tipo	Motor	Seleção de Fontes							
		TPWSS	Avg RWSS	Min RWSS	Max RWSS	Font. Dist.	Seletividade	FP	Red.
Oráculo	RSA	—	—	—	—	—	—	—	—
	FedShop-Go	178,82	1,22	1,00	1,35	22,70	75,8%	0,00	2,20
	PyFedX	104,90	1,10	1,00	1,17	19,59	66,6%	0,00	0,48
Implementação	PyFedX++	161,85	1,22	1,00	1,35	22,70	75,8%	0,00	1,18
	FedX	4,62	0,13	0,08	0,15	2,31	7,7%	0,00	0,00
Baseline	SPLendid	—	—	—	—	—	—	—	—
	SemaGrow	—	—	—	—	—	—	—	—
Metadados									

Tabela 4: Métricas de seleção de fontes por motor; Verde indica melhor global da coluna.

Consulta	FedShop-Go	FedX	RSA	SPLENDID	SemaGrow	PyFedX	PyFedX++
q01	0,44	0,14	2,31	14,74	64,56	T/O	4,27
q02	T/O	3,37	64,97	T/O	T/O	T/O	T/O
q03	4,48	4,37	26,98	14,83	1,09	T/O	7,69
q04	0,11	3,52	2,20	28,52	66,75	T/O	2,89
q05	T/O	3,27	T/O	T/O	62,49	T/O	T/O
q06	0,15	3,10	2,23	1,45	64,59	62,83	0,35
q07	0,13	1,08	1,93	48,46	60,98	3,04	3,90
q08	0,10	0,82	1,88	3,04	61,58	1,46	2,09
q09	0,01	0,41	1,81	1,04	62,46	0,10	0,12
q10	0,05	0,70	1,89	3,86	65,86	0,73	1,38
q11	0,03	0,51	1,95	1,10	62,84	0,20	0,35
q12	0,08	0,82	1,94	11,67	63,25	4,36	3,91

Tabela 5: < 2s rápido, 2–15s médio, 15–60s lento, > 60s / T/O crítico

[link para aplicação](#)

* Válido somente para apresentação em aula.

3. Conclusão

Comparação	Métrica	Δ mediana	W	P-ajustado	n
FedShop-Go $\times$ FedX	TPWSS	+78,00	0,00	0,000035	26
FedShop-Go $\times$ SemaGrow	Tempo	-63,47	10,00	< 0,000001	40
FedShop-Go $\times$ SPLENDID	Tempo	-5,18	0,00	< 0,000001	40
FedShop-Go $\times$ RSA	Tempo	-1,77	0,00	< 0,000001	40
FedShop-Go $\times$ PyFedX	Tempo	-1,25	0,00	< 0,000001	29
FedShop-Go $\times$ RSA	Planejamento	-1,11	0,00	< 0,000001	40
FedShop-Go $\times$ RSA	Seleção	-1,01	0,00	< 0,000001	40
FedShop-Go $\times$ FedX	Seleção	-0,25	0,00	< 0,000001	26
FedShop-Go $\times$ PyFedX	Seleção	-0,25	0,00	< 0,000001	29
FedShop-Go $\times$ FedX	Recall	+1,00	0,00	0,000008	40
FedShop-Go $\times$ SemaGrow	Recall	0,00	0,00	0,000366	40
FedShop-Go $\times$ FedX	Avg RWSS	+1,00	3,00	0,000004	26

Tabela 6: *Testes de Wilcoxon pareado, FedShop-Go $\times$ demais motores ($\alpha = 0.05$); P-ajustado com Holm.*

Resultado principal

FedShop-Go obteve o menor tempo médio nas execuções concluídas, com vantagem estatística sobre os motores avaliados.

Corretude

As respostas concluídas preservaram precisão, revocação e F1 máximos, sem *mismatch* agregado.

Limite atual

A seleção de fontes ainda é ampla: FedX seleciona menos fontes, mas perde revocação neste recorte.

Gargalos

q02 e q05 expõem o custo de resultados intermediários grandes e continuam como casos críticos de *timeout*.

4. Trabalhos Futuros

Linha	Próximo passo
Seleção de fontes	Reduzir TPWSS e fontes irrelevantes com metadados, estatísticas VoID ou catálogos de predicados.
q02 / q05	Aplicar <i>filter pushdown</i> e <i>bind joins</i> mais seletivos para diminuir resultados intermediários.
Planejamento	Adicionar ordenação baseada em custo e cardinalidade, aproximando o motor de técnicas usadas por FedX, SPLENDID e CostFed.
Escala FedShop	Executar o workload completo: 10 batches, 10 instâncias e federações de 20 a 200 membros.
Adaptação	Investigar execução adaptativa inspirada no ANAPSID, sem esperar pela seleção completa antes de iniciar a consulta.

Referências

- (1) Acosta, M.; Vidal, M.-E.; Lampo, T.; Castillo, J.; Ruckhaus, E. ANAPSID: An Adaptive Query Processing Engine for SPARQL Endpoints. Em *The Semantic Web – ISWC 2011*; Lecture Notes in Computer Science; Springer, 2011; Vol. 7031, pp 18–34. https://doi.org/10.1007/978-3-642-25073-6_2.
- (2) Dang, M.-H.; Aimonier-Davat, J.; Molli, P.; Hartig, O.; Skaf-Molli, H.; Crom, Y. L. FedShop: A Benchmark for Testing the Scalability of SPARQL Federation Engines. Em *The Semantic Web – ISWC 2023*; Lecture Notes in Computer Science; Springer, 2023; Vol. 14266, pp 285–301. https://doi.org/10.1007/978-3-031-47243-5_16.
- (3) Saleem, M.; Potocki, A.; Soru, T.; Hartig, O.; Ngomo, A.-C. N. CostFed: Cost-Based Query Optimization for SPARQL Endpoint Federation. Em *Proceedings of the 14th International Conference on Semantic Systems (SEMANTiCS 2018)*; Procedia Computer Science; Elsevier, 2018; Vol. 137, pp 163–174. <https://doi.org/10.1016/j.procs.2018.09.016>.
- (4) Schwarte, A.; Haase, P.; Hose, K.; Schenkel, R.; Schmidt, M. FedX: Optimization Techniques for Federated Query Processing on Linked Data. Em *The Semantic Web – ISWC 2011*; Lecture Notes in Computer Science; Springer, 2011; Vol. 7031, pp 601–616. https://doi.org/10.1007/978-3-642-25073-6_38.
- (5) Görlitz, O.; Staab, S. SPLENDID: SPARQL Endpoint Federation Exploiting VOID Descriptions. Em *Proceedings of the Second International Workshop on Consuming Linked Data (COLD 2011)*; CEUR Workshop Proceedings; CEUR-WS.org, 2011; Vol. 782, pp 13–24.
- (6) Charalambidis, A.; Troumpoukis, A.; Konstantopoulos, S. SemaGrow: Optimizing Federated SPARQL Queries. Em *Proceedings of the 11th International Conference on Semantic Systems (SEMANTiCS 2015)*; ACM, 2015; pp 121–128. <https://doi.org/10.1145/2814864.2814886>.